CS 584: Context-Aware Seq2Seq Prompt Prediction

Eli Reinhold*, Stephen Radice*, Christopher Kaldas*
*Stevens Institute of Technology
ereinhol@stevens.edu, sradice@stevens.edu, ckaldas@stevens.edu
Spring 2026

Abstract

Prompt steering and understanding user intent are increasingly important problems in large language models (LLMs). By understanding how to manipulate LLMs, our capabilities using the tool will increase. This project explores how to predict the original user prompt from an AI-generated response. We propose a context-aware sequence-to-sequence approach using a fine-tuned T5 transformer model. It uses ChatGPT responses as input and predicts the most likely prompt. We created 3 models: a baseline model, one that filtered noisy programming-related data during preprocessing, and one that incorporated conversational history and filtered data. We tested various methods to improve model predictions. We evaluated the models using ROUGE-L scores across conversations of varying depth. Experimental results show that the model performs well on short-context conversations and outperforms baseline ChatGPT-4o-mini predictions. However, performance decreases as dialogue history grows longer, suggesting that excessive context introduces noise and reduces the model's ability to identify relevant intent. The model that only filtered the data performed better than the model that included context. These findings demonstrate both the feasibility and limitations of reverse prompt prediction using transformer-based architectures. Future work includes improving context summarization methods to reduce noise from long-range conversational information and upgrading our model to a larger-scale model, such as T5-Base/Llama variants.

1 Introduction

With the development of Large Language Models (LLMs) such as ChatGPT, AI chatbots have become integrated into the daily lives and workflow of many people. They are transforming areas such as education, business, programming, and daily communication. As LLMs are blending into our daily lives, prompt engineering has become increasingly more important. Small differences in prompt wording can significantly change the quality, accuracy, and style of generated responses. Knowing how to maximize the capabilities of LLMs has become an important skill. Because of this, understanding the relationship between prompts and outputs has become a major area of interest in natural language processing (NLP) research.

Most existing NLP systems focus on predicting responses from user prompts. However, doing the reverse—generating the original prompt from an AI-generated response—has received far less attention. Solving this problem could improve prompt steering and understanding of how users interact with language models. Reverse prompt prediction is challenging because multiple prompts can produce similar responses, making it difficult for a model to predict the user's original input.

This project explores how to predict the original user prompt from an AI-generated response. We propose a context-aware sequence-to-sequence approach using a fine-tuned T5 transformer model. The model uses ChatGPT responses as input and predicts the most likely prompt. We created three models: a baseline model, one that filtered noisy programming-related data during preprocessing, and one that incorporated conversational history along with filtered data. We tested several approaches to improve model predictions and evaluated performance using ROUGE-L scores across conversations of varying depth.

Experimental results show that the model performs well on short-context conversations and outperforms baseline ChatGPT-4o-mini predictions. However, performance decreases as dialogue history grows longer, suggesting that

excessive context introduces noise and reduces the model's ability to identify relevant intent. Additionally, the model that only filtered the data performed better than the model that included conversational context.

2 Related Work

Several works have explored reversing input-output mappings and generating questions from conversational context. In 2017, Du et al. explored automatic question generation for reading comprehension, framing the task as a sequence-to-sequence problem that maps text passages to natural questions [2]. They found that neural models outperform rule-based approaches in both complexity and naturalness. In 2024, Dong et al. proposed a context-aware and user intent-aware follow-up question generation method [1]. They found that in multi-turn conversational settings, incorporating broader dialogue history can introduce noise and worsen performance. In 2025, Miehling et al. studied prompt steerability in LLMs, finding that small differences in prompt wording can significantly shift model behavior [4]. Similarly, our work predicts the original user prompt in a multi-turn setting, and we also observed that adding conversational history as context degrades model performance. Our findings also highlight the sensitive relationship between prompts and LLM outputs.

Since its introduction in 2019, the T5 model has been used to solve many problems. Raffel and the Google AI team introduced T5, a unified transformer framework pre-trained on the C4 dataset in 2019 [5]. They demonstrated that increasing model and data size correlates with strong results across various NLP tasks. Recent work has demonstrated the versatility of T5 and its uses across diverse areas. Zhuang et al. fine-tune T5 for text ranking [7], while Yermakov et al. fine-tune T5 for biomedical data-to-text generation [6], both demonstrating that T5 can be effectively adapted to new tasks beyond its original objectives. Like previous research, we fine-tune a T5 model. However, unlike them, we adapt it for reverse prompt prediction, filtering out programming noise and incorporating a two-turn history context window.

3 Methodology

We frame reverse prompt prediction as a sequence-to-sequence (seq2seq) task: given a ChatGPT response (and, optionally, prior conversational context), generate the most likely user prompt that produced it. Formally, let r denote a model response and p denote the corresponding user prompt. Our goal is to learn a mapping $f : r \mapsto \hat{p}$ such that $\hat{p}$ is as close as possible to the ground-truth prompt p under a lexical similarity metric. All three models are built on T5-small, a compact encoder-decoder transformer pre-trained on the C4 corpus using a denoising span-corruption objective. We fine-tune it using the task prefix `predict prompt:` prepended to each input, following the standard T5 task-conditioning convention.

3.1 Architecture and Training

Each input answer is tokenized to a maximum length of 512 tokens, and target prompts are tokenized to a maximum of 128 tokens. Padding tokens in the target sequence are replaced with -100 so that the cross-entropy loss does not penalize the model for padding positions. All models are trained with the AdamW optimizer at a learning rate of 5×10^{-5} and a batch size of 8 on a GPU, with gradient clipping at a norm of 1.0 to stabilize training. At the end of each epoch, validation loss is computed on the held-out validation set; the checkpoint with the lowest validation loss is saved as the best model. Beam search with 4 beams and early stopping is used at inference time to generate predicted prompts.

3.2 Model v1 — Initial Model

The first model is trained on the cleaned dataset without any additional filtering or contextual information. The raw answer text is provided as input with the `predict prompt:` prefix, and the model is trained for a fixed 5 epochs.

3.3 Model v2 — Filtered Model

The second model introduces a preprocessing step designed to remove examples the model consistently struggled with. Specifically, rows where the answer consisted primarily of code blocks (identified by `[CODE_BLOCK_N]` placeholders) are removed, as are answers that become too short after code removal. Prompts fewer than 3 words or longer than 50 words, and answers longer than 400 words, are also filtered out. This filtering is intended to reduce noise from formatting-heavy and domain-specific content. The model is otherwise trained identically to v1 (5 fixed epochs, same architecture and hyperparameters).

3.4 Model v3 — Context-Aware Model

The third model extends v2 by incorporating conversational history as additional input. For each turn in a conversation, up to the two most recent prior turns (each consisting of the previous user prompt and the corresponding assistant answer) are prepended to the current answer, forming a context-aware input of the form:

```
predict prompt: User: <prev_prompt> Assistant: <prev_answer> ... Current
answer: <current_answer>
```

Conversations are grouped by their URL identifier and sorted by turn index before applying this windowing procedure. To prevent overfitting and allow the model to adaptively determine training duration, v3 replaces the fixed 5-epoch loop with early stopping: training halts when validation loss fails to improve for 3 consecutive epochs.

3.5 Baseline

As a point of comparison, we construct a zero-shot baseline using GPT-4o-mini. For each test example, we query the model with the prompt *"Given a GPT model outputted: {answer}, predict the prompt."* and score the result using ROUGE-L against the reference prompt. This baseline is evaluated on every 100th row of the filtered dataset to manage API cost.

4 Experimental Setup

4.1 Data

The dataset consists of 8,058 real user conversations with ChatGPT from the DevGPT dataset. Each sample includes metadata such as the user's name, the date of the conversation, the user's prompt, the model's response, the turn index within the conversation (`conv_idx`), and a link to the conversation thread. This multi-turn structure provides rich context for understanding how users interact with the model across a dialogue.

Before training, several preprocessing steps were applied to improve data quality. First, all non-English samples were removed, as the model is designed to operate in English and multilingual entries would introduce noise. Second, code-heavy responses were handled carefully: responses containing three or more `[CODE_BLOCK_N]` placeholders (representing extracted code snippets) were either removed or had those placeholders stripped via regex substitution, depending on the model version. Any response that became shorter than 10 words after code removal was also discarded, as the remaining text was unlikely to carry sufficient semantic signal for the model to recover the original prompt.

Additional length-based filters were applied to focus the model on tractable examples. Prompts shorter than 3 words were removed as they lack sufficient content, and prompts longer than 50 words were removed as they are atypically long and tend to be complex multi-part requests that are difficult to recover from a response alone. Responses longer than 400 words were similarly excluded, as they often contain extensive elaboration that obscures the connection to the original prompt.

After filtering, the dataset was split into training, validation, and test sets using a 70/15/15 ratio. For the context-aware model (v3), conversations were grouped by their URL identifier and sorted by turn index, and a sliding two-turn history window was constructed for each sample before splitting.

4.2 Evaluation Metrics

Model performance is measured using the ROUGE family of metrics [3], which evaluate the degree of lexical overlap between the predicted prompt and the reference (ground-truth) prompt. Three variants are computed:

- **ROUGE-1** measures the overlap of individual unigrams between the prediction and reference.
- **ROUGE-2** measures the overlap of consecutive bigrams.
- **ROUGE-L** measures the length of the longest common subsequence, capturing word-order similarity in addition to word identity.

All scores are reported as F1 measures and computed with stemming enabled to reduce sensitivity to morphological variation. ROUGE-L is the primary metric for model comparison, as it provides the most holistic measure of how closely the predicted prompt matches the original in both content and phrasing.

In addition to the T5 models, we evaluate a GPT-4o-mini zero-shot baseline on the same ROUGE-L metric to provide a reference point that reflects what a capable LLM can recover without any task-specific fine-tuning.

4.3 Comparison Methods

We compare the following methods:

- **Model v1 (Baseline T5-small):** Fine-tuned T5-small without data filtering or context
- **Model v2 (Filtered T5-small):** Fine-tuned T5-small with code-heavy and length-outlier examples removed
- **Model v3 (Context-Aware T5-small):** Fine-tuned T5-small with filtered data and a two-turn conversational history window
- **GPT-4o-mini Zero-Shot Baseline:** A zero-shot prompt inversion query to GPT-4o-mini

5 Results

We evaluate all three T5 variants and the GPT-4o-mini zero-shot baseline on the same test split using ROUGE-1, ROUGE-2, and ROUGE-L F1 scores. We treat ROUGE-L as the primary comparison metric because it captures both word overlap and word order. Table 1 reports the aggregate scores, and Figure 1 visualizes the ROUGE-L comparison.

Table 1: Aggregate ROUGE F1 scores on the test set. The filtered model (v2) achieves the best performance across all three metrics, while every fine-tuned T5 variant outperforms the GPT-4o-mini zero-shot baseline.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Model v1 (initial)	0.265	0.118	0.231
Model v2 (filtered)	0.341	0.169	0.302
Model v3 (context-aware)	0.309	0.152	0.270
GPT-4o-mini (zero-shot baseline)	0.214	0.082	0.179

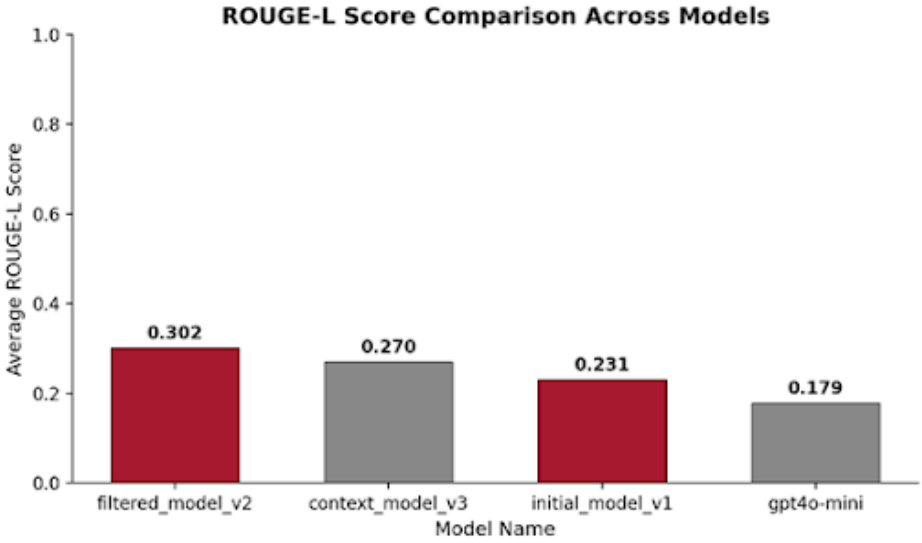

Figure 1: Average ROUGE-L score across the four evaluated systems. The filtered model (v2) is the strongest configuration; adding conversational context (v3) produces a small but consistent drop. All fine-tuned T5 variants exceed the GPT-4o-mini zero-shot baseline.

Three findings stand out. First, every fine-tuned T5 variant beats the GPT-4o-mini zero-shot baseline by a wide margin on ROUGE-L, confirming that task-specific fine-tuning on a small encoder–decoder substantially outperforms a much larger general-purpose LLM on reverse prompt prediction. Second, the filtered model (v2) is the best overall model, improving ROUGE-L by a substantial margin over the unfiltered initial model (v1). Third, adding two turns

of conversational history (v3) does not help; v3 trails v2 slightly, despite using the same filtering pipeline. We explore this counter-intuitive result in more depth below.

5.1 Effect of Conversation Depth

To stress-test the context-aware model, we measure ROUGE on every test example as a function of `conv_index`, the turn position of the current prompt within its full conversation. Because the v3 sliding window only retains the two most recent turns, examples with `conv_index` > 2 are missing earlier dialogue history that the user may implicitly reference. Figure 2 plots the resulting trend.

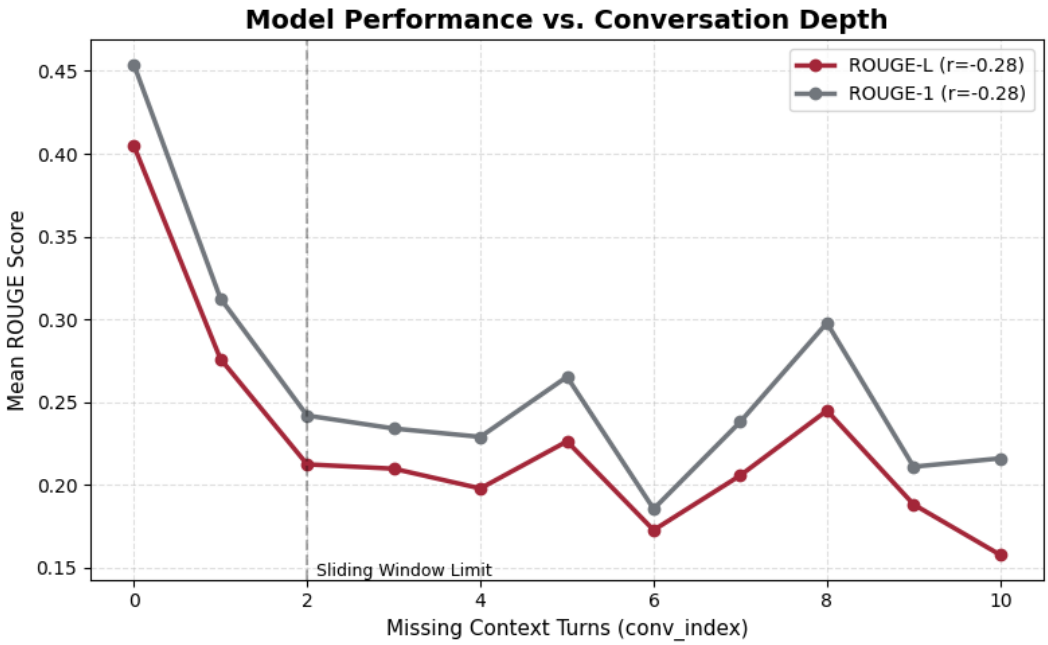

Figure 2: Mean ROUGE-1 and ROUGE-L score versus `conv_index` (the turn position of the prompt). Both metrics are negatively correlated with depth (Pearson $r = -0.28$). Performance is highest on the opening turn (`conv_index` $= 0$) and falls sharply once the prompt sits beyond the model's two-turn sliding window.

The trend is clear and monotonic in the first several turns. At `conv_index` $= 0$, where the prompt has no missing prior context, the model reaches a mean ROUGE-L of roughly 0.41—nearly the level of strong sentence-paraphrase systems. ROUGE-L drops to about 0.27 by the second turn and stabilizes in the 0.16–0.25 range as the conversation deepens further. Once the prompt is no longer covered by the two-turn sliding window (the dashed vertical line in Figure 2), additional context only adds tokens that the encoder cannot easily map back to user intent.

5.2 Effect of Code Content

We also split the v3 test set into prompts that contain code constructs (identifiers, keywords like `def`/`class`/`import`, braces, and comparison operators) and prompts that do not. Code-containing prompts produce a mean ROUGE-L of 0.240, while the 460 non-code prompts reach 0.283. The 4-point gap shows that even after filtering responses dominated by code blocks, residual programming vocabulary still degrades recovery. Programming answers tend to share phrasing across very different prompts, which makes the reverse mapping harder.

5.3 Qualitative Examples

Beyond aggregate scores, we inspect representative predictions to understand where the model succeeds and where it fails. Table 2 reproduces three examples drawn from low, medium, and high performance buckets.

Table 2: Representative predictions at three score levels. The high-scoring case is a self-contained question that the answer trivially exposes. The medium case is a true paraphrase. The low case shows the popular deep-context failure model, which echoes a related earlier topic instead of decoding the elliptical follow-up.

	Low Performance (ROUGE-L: 0.25)	Medium Performance (ROUGE-L: 0.51)	High Performance (ROUGE-L: 0.90)
Turn	8 (Deep Context)	4 (Mid Context)	1 (Start)
Actual Prompt	"Can you do the same for CAVA?"	"How do I change my primary monitor on common Oses?"	"What is EDI 945?
Model Prediction	"Summarize news about Sweetgreen"	"How do I set a primary display on the three most common operating systems?"	"What is the EDI 945?"

Inspecting test predictions with ROUGE-L ≤ 0.01 reveals two recurring failure patterns. The first is the deep-context drift shown above, where the model substitutes a topically similar prompt from earlier in the conversation. The second is a fallback to a short, generic question (e.g., "Can you explain the conversion process?") when the reference prompt is itself a short conversational reaction such as "That was wrong. What happened?" In both cases the model produces fluent text; it is the alignment with the true user intent that breaks down.

Figure 3 plots the full distribution of ROUGE-L scores on the v3 test split. The distribution is heavily right-skewed: most predictions cluster in the 0.10–0.35 range, but a meaningful tail of predictions exceeds 0.8, including 20 perfect matches.

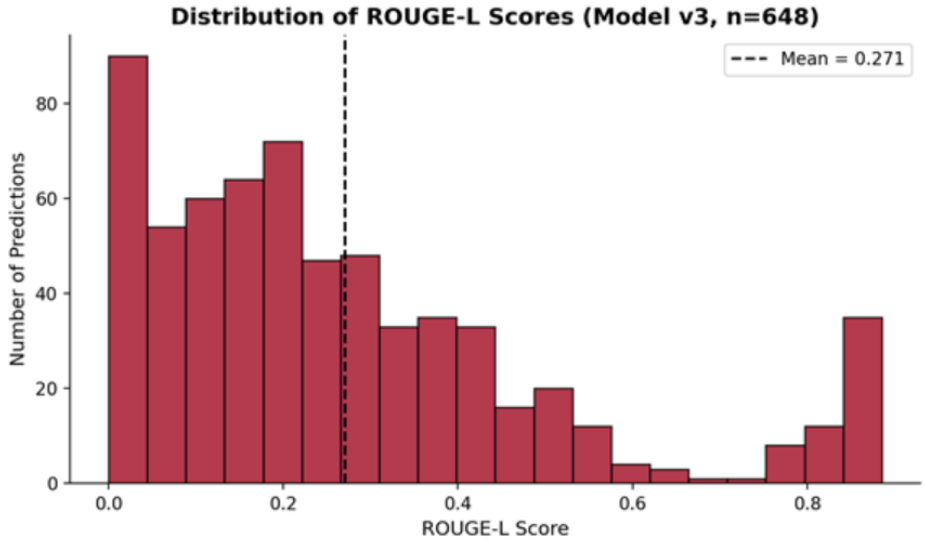

Figure 3: Distribution of ROUGE-L scores for the context-aware model (v3) across the test set. The right tail captures self-contained factual prompts that the answer fully exposes, and the zero-mass on the left captures the deep-context and conversational-reaction failures.

6 Discussion

Our results suggest that a small, fine-tuned encoder–decoder transformer is a surprisingly good fit for reverse prompt prediction, but only when the input is kept clean and the task is framed at the right granularity. The fine-tuned T5-small models recover the original user prompt with a mean ROUGE-L of up to 0.30, clearly better than the GPT-4o-mini zero-shot baseline, despite the baseline having orders of magnitude more parameters. This indicates that task-specific supervised fine-tuning, even on roughly 4,000 examples, can outperform a frontier LLM on a task it was not specifically trained for.

The most informative result is the failure of the context-aware model (v3) to improve over the filtered model (v2). The intuition behind v3—that prior turns should clarify the current prompt—is reasonable in principle, but in practice the additional turns flood the encoder with content that is only weakly related to the user's most recent intent. The encoder must learn to attend to the current answer while suppressing everything that came before, and a 60-million parameter T5-small does not have the capacity to do this reliably. The `conv_index` trend in Figure 2 confirms the same effect from a different angle: even with windowed context, the model degrades as the prompt drifts further from the start of the conversation, because earlier turns the user implicitly references are now outside the window. This mirrors prior work by Dong et al. [1], which similarly found that uncurated dialogue history hurts more than it helps.

The project builds on three key capabilities. First, the pretrained T5 checkpoint, which already encodes a strong prior over English text and the seq2seq formulation. Second, GPT-4o-mini's zero-shot instruction-following, used as a meaningful comparison baseline without any task-specific tuning. Third, beam search and length penalties at decoding time, which let the model produce coherent question-shaped outputs rather than statistical noise. T5-small simply does not do the kind of multi-step intent inference that a deep conversation requires, which is why so many low-scoring failures are fluent but discuss a completely irrelevant topic.

We also note weaknesses worth being explicit about. The model handles conversational follow-ups poorly because they depend on long-range correlations. ("Can you do the same for CAVA?" depends on knowing what was done for the previous restaurant). Sliding the context window outward is not the right fix, since longer windows hurt more than they help. The model also defaults to generic prompts when uncertain—something ROUGE captures only partially, since a generic prompt can share words with the reference by coincidence and inflate the score without actually capturing intent.

7 Conclusion

This project investigated reverse prompt prediction: given a ChatGPT response, recover the user prompt that produced it. We framed the task as sequence-to-sequence modeling and fine-tuned three T5-small variants with an initial model, a filtered model, and a context-aware model that incorporated two prior dialogue turns. Then, it compared them against a GPT-4o-mini zero-shot baseline on filtered ChatGPT conversations.

All three fine-tuned variants beat the GPT-4o-mini baseline by a wide margin, with the filtered model (v2) reaching the best ROUGE-L of 0.302 versus 0.179 for the baseline. Counter to our initial hypothesis, adding two turns of conversational history (v3) consistently underperformed the same model without context. This confirmed a pattern: ROUGE-L is highest on the first turn of a conversation and falls steadily as the prompt drifts further into the conversation, because each additional turn adds noise the small encoder cannot reliably filter. Aggressive data filtering, by contrast, delivered significant ROUGE-L gain at no additional cost, which confirms that supervision quality is what matters when filtering the data.

For future work, we recommend two main directions. First, instead of feeding the model the full text of prior conversational turns, future implementations will extract only the keywords or a short summary of prior context before passing it to the encoder. This would let the model benefit from dialogue history without the input length growing large enough to drown out the current answer, which we identified as the main reason our context aware model underperformed. Second, the failure patterns we observed (lost confidence on follow-up prompts, fallback to older conversation outputs) are happening because of insufficient model capacity, so scaling the backbone of our model from T5-small to a larger model such as T5-base or a Llama variant would likely produce substantial gains on deeper conversational turns.

References

[1] Dong et al. Context-aware and user intent-aware follow-up question generation (CA-UIA-QG): Mimicking user behavior in multi-turn setting. In *Amazon Science*, 2024.

[2] Xinya Du, Junru Shao, and Claire Cardie. Learning to ask: Neural question generation for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017. doi: 10.48550/arXiv.1705.00106.

[3] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries, 2004.

[4] Miehling et al. Evaluating the prompt steerability of large language models. *arXiv preprint arXiv:2411.12405*, 2025. doi: 10.48500/arXiv.2411.12405.

[5] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv preprint arXiv:1910.10683*, 2023. doi: 10.48500/arXiv.1910.10683.

[6] Ruslan Yermakov, Nicholas Drago, and Alan Mishler. Biomedical data-to-text generation via fine-tuning transformers. In *Proceedings of the 14th International Conference on Natural Language Generation*, pages 364–370, 2021. doi: 10.18653/v1/2021.inlg-1.40.

[7] Honglei Zhuang, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. RankT5: Fine-tuning T5 for text ranking with ranking losses. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2308–2313, 2023. doi: 10.1145/3539618.3592047.